



XỬ LÝ ẢNH - NHÓM 5

Nhận dạng chữ số viết tay và hình dạng đơn giản bằng mạng neural CNN

Thành viên nhóm:

Đỗ Đức Cảnh - B22DCCN086

Trần Quang Huy - B22DCCN397

Trần Quang Huy - B22DCCN398

TỔNG QUAN HỆ THỐNG

1. Bài toán

- Xây dựng hệ thống có khả năng nhận dạng chữ số viết tay (0–9), các chữ cái và các hình dạng cơ bản (tam giác, vuông, tròn, chữ nhật...) từ ảnh đầu vào.
- Áp dụng kết hợp kỹ thuật xử lý ảnh và mạng nơ-ron tích chập (CNN).

2. Mục tiêu

- Nghiên cứu và áp dụng các kỹ thuật xử lý ảnh tiền xử lý: lọc nhiễu, chuẩn hóa, nhị phân hóa, contour.
- Xây dựng 3 mô hình CNN độc lập cho 3 nhiệm vụ:
 - Nhận dạng chữ số viết tay.
 - Nhận dạng hình dạng cơ bản.
 - Nhận dạng chữ cái
- Phát triển giao diện demo trực quan để người dùng tải ảnh hoặc vẽ trực tiếp để kiểm thử mô hình.

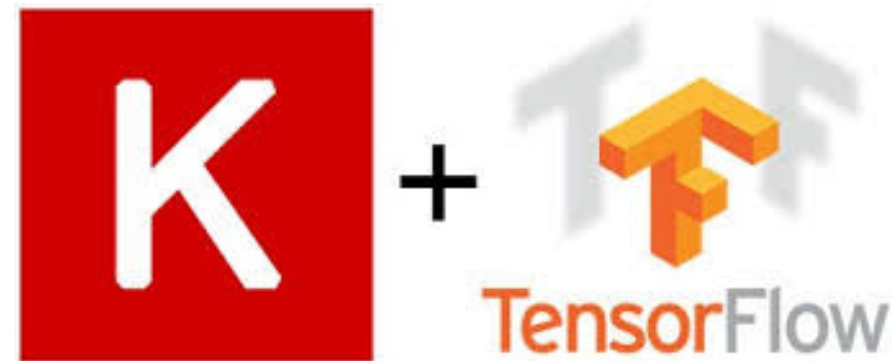
3. Phạm vi thực hiện

- Ảnh đầu vào là ảnh đơn lẻ.
- Chỉ tập trung vào chữ số (0–9), chữ cái và 5–7 loại hình dạng cơ bản.
- Không xử lý video, không nhận dạng nhiều đối tượng trong một ảnh.
- Dữ liệu chủ yếu từ MNIST, tập tự thu thập và dataset hình dạng tự tạo.

CÔNG NGHỆ & CÔNG CỤ



Ngôn ngữ chính để xây dựng toàn bộ hệ thống.



Xây dựng và huấn luyện các mô hình CNN.



OpenCV

Tiền xử lý ảnh (lọc nhiễu, nhị phân, crop, resize).



Xử lý ma trận và dữ liệu ảnh dạng số.

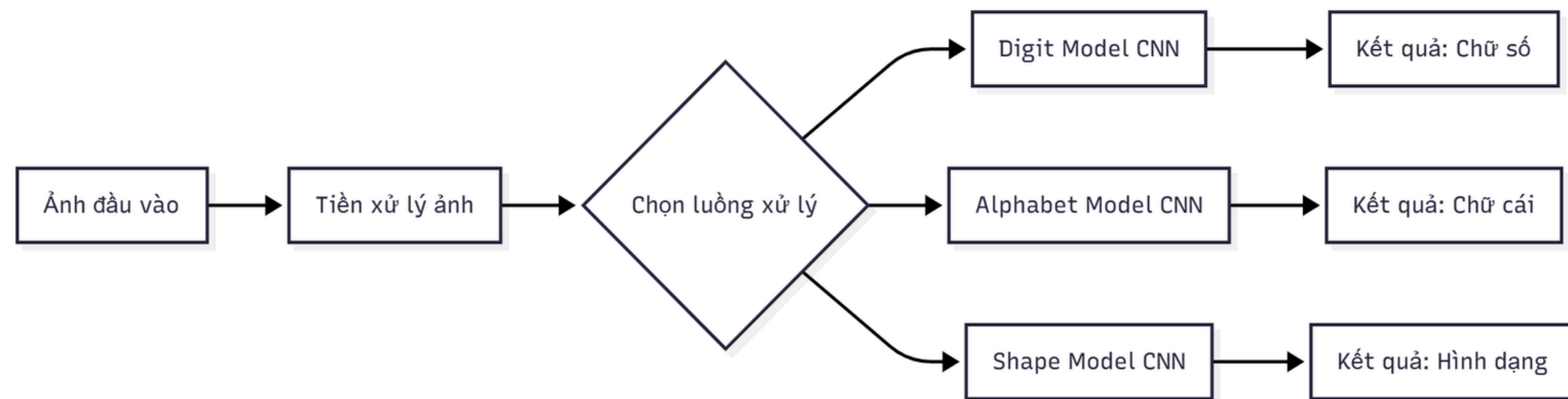


Streamlit

Tạo giao diện demo cho người dùng tương tác.

Kiến trúc hệ thống

- Hệ thống nhận một ảnh bất kỳ chứa chữ số, chữ cái hoặc hình dạng đơn giản.
- Ảnh đầu vào được tiền xử lý: giảm nhiễu, nhị phân hóa, chuẩn hoá kích thước để làm sạch dữ liệu.
- Dựa vào đặc trưng ban đầu, hệ thống chọn nhánh nhận dạng chữ số, chữ cái hoặc nhánh nhận dạng hình dạng.
- Ảnh chuẩn hóa được đưa vào mạng CNN tương ứng để dự đoán lớp cuối cùng.



Tiền xử lý ảnh

Hệ thống áp dụng chuỗi thuật toán xử lý ảnh để chuẩn hóa đầu vào trước khi đưa vào CNN:

1. Chuyển xám (Grayscale)
2. Làm mờ bằng Gaussian Blur
3. Nhị phân hóa bằng thuật toán Otsu
4. Đảo màu (Invert)
5. Crop đối tượng + căn giữa
6. Resize về 28×28

→ Mục tiêu: loại nhiễu – tách nền – giữ nét chính – chuẩn kích thước.



Tiền xử lý ảnh

BƯỚC 1 Grayscale Conversion

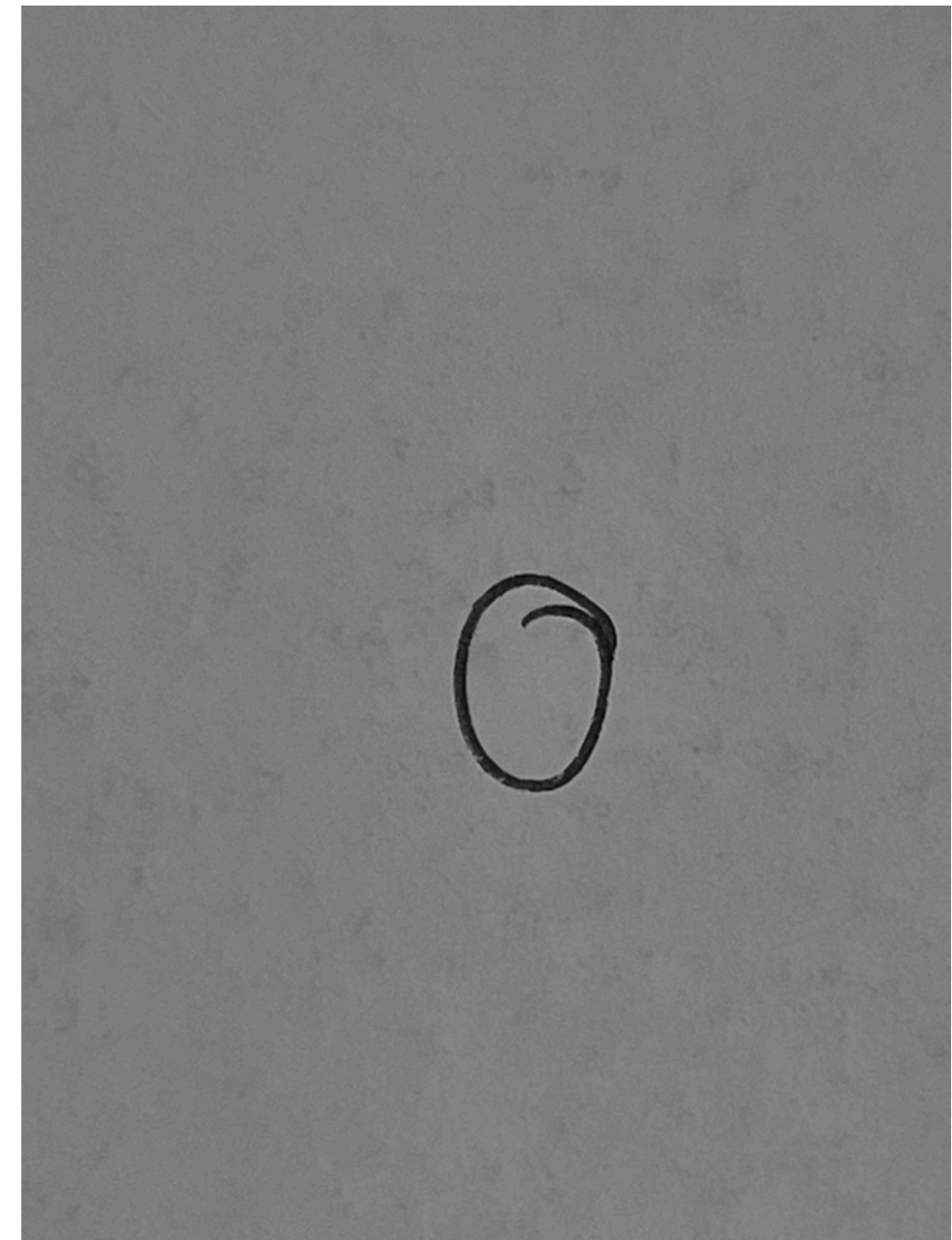
Thuật toán:

RGB → Gray theo công thức y học / thị giác người:

$$Gray=0.299R+0.587G+0.114B$$

Ý nghĩa:

- Giảm số kênh màu từ 3 → 1 → nhanh hơn
- Loại bỏ thông tin không cần thiết
- Giữ lại cường độ sáng – đặc trưng quan trọng cho việc phân biệt nét vẽ



Ảnh gốc sau khi chuyển mức xám

Tiền xử lý ảnh

BƯỚC 2 Gaussian Blur – Làm mờ ảnh

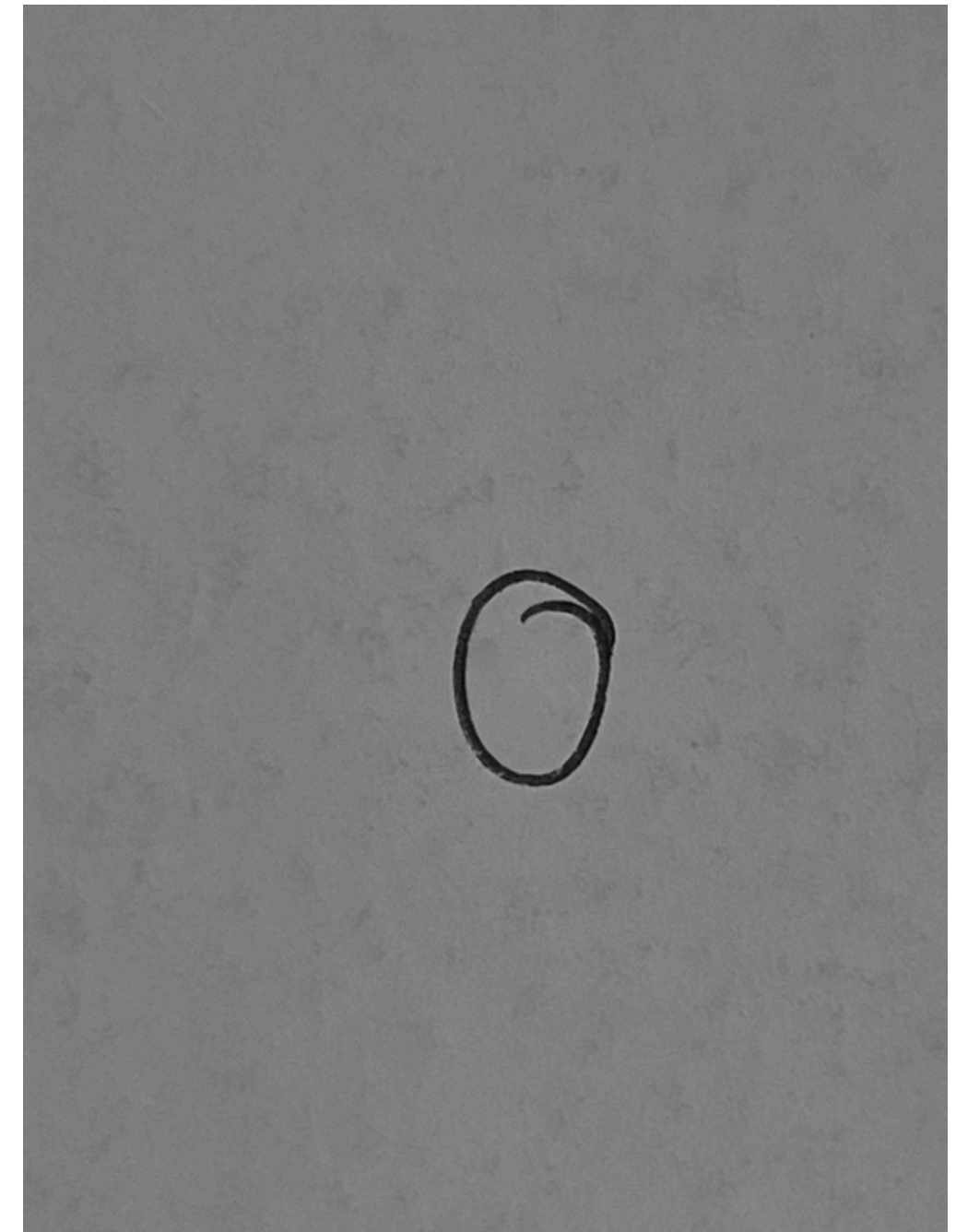
Thuật toán:

Ảnh được convolution với kernel Gaussian:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2) \div (2\sigma^2)}$$

Tác dụng:

- Giảm nhiễu dạng chấm, gợn
- Làm mờ biên → giúp Otsu Threshold chính xác hơn
- Tránh trường hợp threshold bị ảnh hưởng bởi pixel nhiễu



Ảnh sau khi được làm mờ

Tiền xử lý ảnh

BƯỚC 3 Otsu Threshold – Nhị phân hóa tự động

Nguyên lý hoạt động:

Otsu tìm ngưỡng sao cho độ phương sai giữa 2 lớp (foreground & background) là lớn nhất:

$$\sigma_b^2 = \omega_0(\mu_0 - \mu)^2 + \omega_1(\mu_1 - \mu)^2$$

Trong đó:

- ω_0, ω_1 : tỉ lệ pixel nền và vật thể
- μ_0, μ_1 : mức xám trung bình của 2 lớp
- μ : mức xám toàn ảnh

Kết quả:

Ảnh chuyển thành đen – trắng hoàn toàn, tách rõ nét vẽ và nền.

Tại sao dùng Otsu?

- Tự động
- Rất ổn định với ảnh chữ số / hình dạng
- Không bị phụ thuộc ánh sáng môi trường



Ảnh sau khi được phân ngưỡng tự động

Tiền xử lý ảnh

BƯỚC 4 Invert (Đảo màu)

Thuật toán:

$$I' = 255 - I$$

Mục đích:

- Trả ảnh về đúng chuẩn MNIST (nét trắng trên nền đen)
- Giúp CNN học đồng bộ giữa dữ liệu tự thu thập và dataset gốc

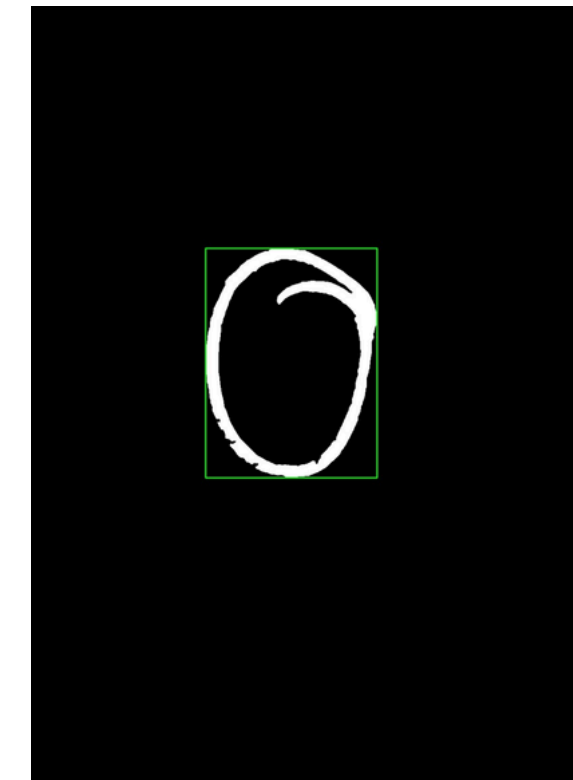
BƯỚC 5&6 Crop + Resize

Thuật toán:

1. Tìm bounding box từ contour lớn nhất và cắt ảnh
2. Đặt đối tượng vào trung tâm khung vuông và thêm padding
3. Sử dụng bilinear interpolation hoặc nearest neighbor.

Mục đích:

- Loại bỏ khoảng trắng dư
- Đảm bảo chữ số/hình nằm giữa → CNN nhận dạng tốt hơn
- Giảm hiện tượng lệch trái/phải gây sai dự đoán
- Chuẩn hóa kích thước input model



Tìm bounding



Ảnh cuối sau tiền xử lý

Mô hình CNN

NHẬN DẠNG CHỮ SỐ - KIẾN TRÚC TỔNG QUAN

- Ảnh đầu vào: 28×28, grayscale
- Tiền xử lý: Normalize, reshape
- Data Augmentation: xoay, dịch, zoom, shear
- 3 khối Convolution:
 - Conv32 + MaxPooling + Dropout
 - Conv64 + MaxPooling + Dropout
 - Conv64 + Dropout
- Fully Connected:
 - Flatten
 - Dense 128 + Dropout 0.5
 - Dense 10 (Softmax)

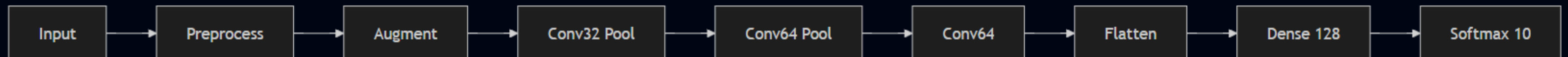
```
model = keras.Sequential([
    # Input layer
    layers.Input(shape=input_shape),

    # Convolutional Block 1
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Dropout(0.25),

    # Convolutional Block 2
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Dropout(0.25),

    # Convolutional Block 3
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same'),
    layers.Dropout(0.25),

    # Fully Connected Layers
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
```



Mô hình CNN

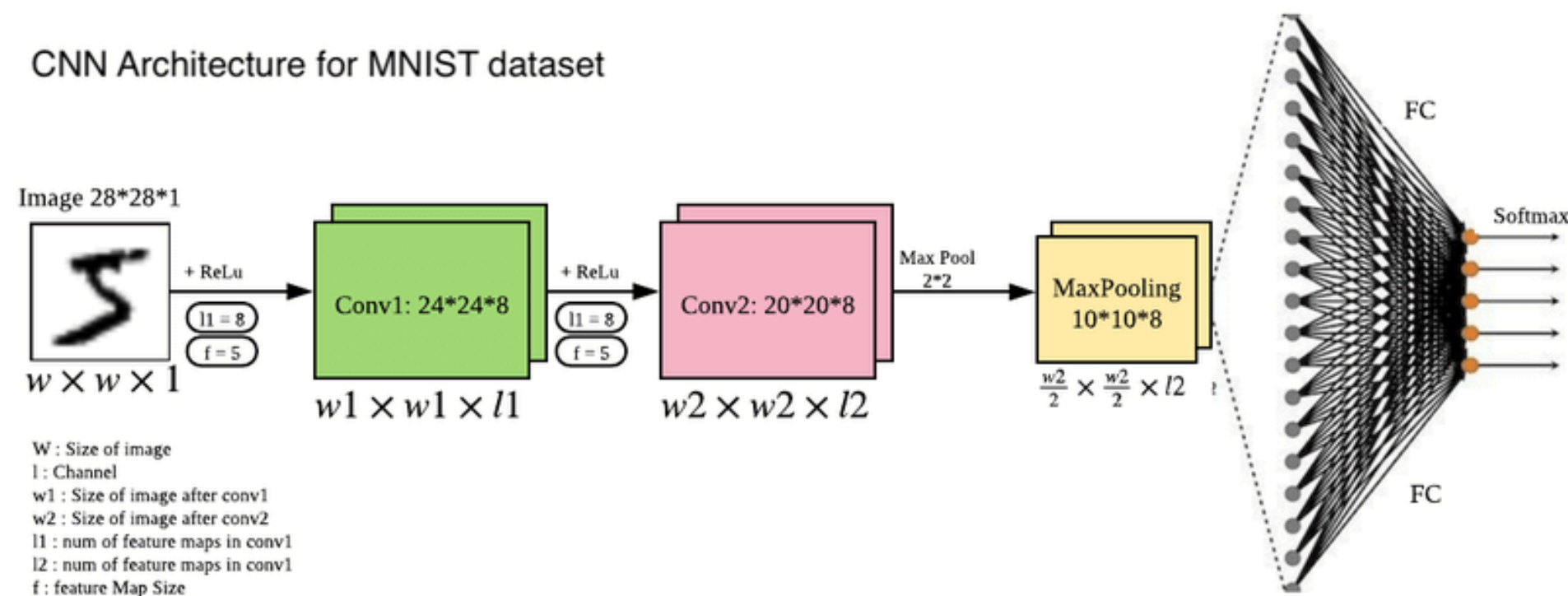
NHẬN DẠNG CHỮ SỐ - HOẠT ĐỘNG

- **Bước 1 – Tiền xử lý:** chuẩn hoá pixel về [0–1], chuyển về $28 \times 28 \times 1$
- **Bước 2 – Tăng cường dữ liệu:** tạo ảnh xoay/ dịch → giúp mô hình nhận dạng chữ viết tay không hoàn hảo
- **Bước 3 – Conv Layers:** trích xuất đặc trưng (cạnh, nét cong, nét xiên)
- **Bước 4 – Pooling:** giảm nhiễu, giữ đặc trưng quan trọng
- **Bước 5 – Dense Layers:** tổng hợp đặc trưng và phân loại
- **Bước 6 – Softmax:** xuất xác suất 10 chữ số (0–9)

MÔ HÌNH GIÚP:

- Tự động học đặc trưng hình ảnh
- Chịu được lệch, xoay, méo
- Dùng filters để nhận dạng nét cong, nét xiên
- Cho độ chính xác rất cao ($> 98\%$)

CNN Architecture for MNIST dataset

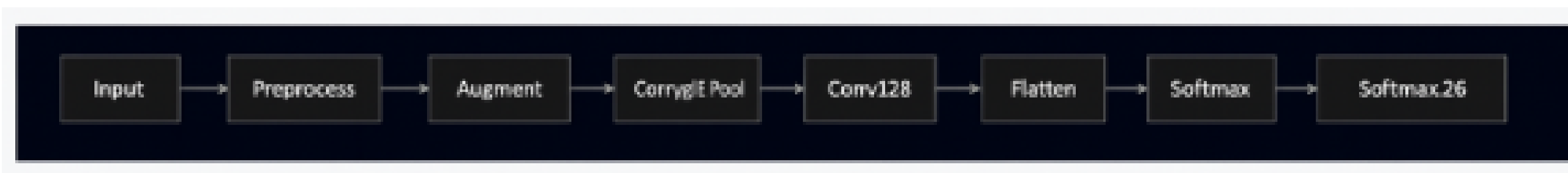


Mô hình CNN

NHẬN DẠNG CHỮ VIẾT TAY A-Z - KIẾN TRÚC TỔNG QUAN

- Ảnh đầu vào: 28×28, grayscale
- Tiền xử lý: normalize pixel về [0–1], reshape → (28,28,1)
- Không dùng Data Augmentation
- 3 khối Convolution:
 - Conv32 + MaxPooling
 - Conv64 + MaxPooling
 - Conv128 + MaxPooling
- Fully Connected:
 - Flatten
 - Dense 128 + Dropout 0.5
 - Dense 26 (Softmax – A → Z)

```
model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation="softmax"), # 26 lớp A-Z
])
```



Mô hình CNN

NHẬN DẠNG CHỮ VIẾT TAY A-Z - HOẠT ĐỘNG

- Bước 1 – Tiền xử lý: chuẩn hoá ảnh về $[0-1]$, reshape về $(28 \times 28 \times 1)$
- Bước 2 – Conv Layers: trích xuất đặc trưng (nét cong, nét thẳng, góc)
- Bước 3 – MaxPooling: giảm nhiễu, giữ đặc trưng quan trọng
- Bước 4 – Flatten: chuyển tensor thành vector
- Bước 5 – Dense 128: học đặc trưng tổng hợp
- Bước 6 – Softmax: dự đoán 26 ký tự ($A \rightarrow Z$)

CHỨC NĂNG CHÍNH CỦA MÔ HÌNH

- Tự học đặc trưng hình ảnh không cần bạn thiết kế thủ công
- Nhận dạng chữ viết tay kể cả khi nét không hoàn hảo
- Xử lý tốt nhiễu, độ lệch, độ mờ
- Độ chính xác cao khi dữ liệu đủ lớn

Mô hình CNN

NHẬN DẠNG HÌNH HỌC – KIẾN TRÚC MÔ HÌNH

- Ảnh đầu vào: 64×64, grayscale
- Tiền xử lý: Normalize (0–1), reshape
- Data Augmentation: xoay, dịch, zoom, chỉnh contrast

3 Khối Convolution:

- Block 1: Conv32 → BN → Conv32 → BN, MaxPooling, Dropout 0.25
- Block 2: Conv64 → BN → Conv64 → BN, MaxPooling, Dropout 0.25
- Block 3: Conv128 → BN → Conv128 → BN, MaxPooling, Dropout 0.25

Fully Connected:

- Flatten
- Dense 256 + BatchNorm + Dropout 0.5
- Dense 128 + BatchNorm + Dropout 0.5
- Dense N lớp (Softmax) – tùy theo số class trong dataset

```
model = models.Sequential([
    layers.Input(shape=(IMG_SIZE, IMG_SIZE, 1)),

    # Block 1
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Block 2
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Block 3
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Dense layers
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(NUM_CLASSES, activation='softmax')
])
```

Mô hình CNN

NHẬN DẠNG HÌNH HỌC – QUY TRÌNH HOẠT ĐỘNG

- Bước 1 – Tiền xử lý: chuẩn hóa pixel 0–1, chuyển về kích thước $64 \times 64 \times 1$
- Bước 2 – Tăng cường dữ liệu: tạo ảnh xoay/dịch/zoom/contrast → giúp mô hình bền vững hơn
- Bước 3 – Convolution Layers: trích xuất biên, cạnh cong, texture, hình khối
- Bước 4 – Pooling: giảm chiều dữ liệu, giữ đặc trưng quan trọng
- Bước 5 – Dense Layers: tổng hợp đặc trưng và phân loại
- Bước 6 – Softmax: dự đoán xác suất cho từng lớp hình (triangle, circle, square, ...)

MÔ HÌNH GIÚP:

- Tự động học đặc trưng hình học
- Ổn định khi ảnh bị xoay/dịch/zoom
- Nhận dạng cạnh, góc, đường cong
- Độ chính xác cao (> 95% nếu dataset tốt)

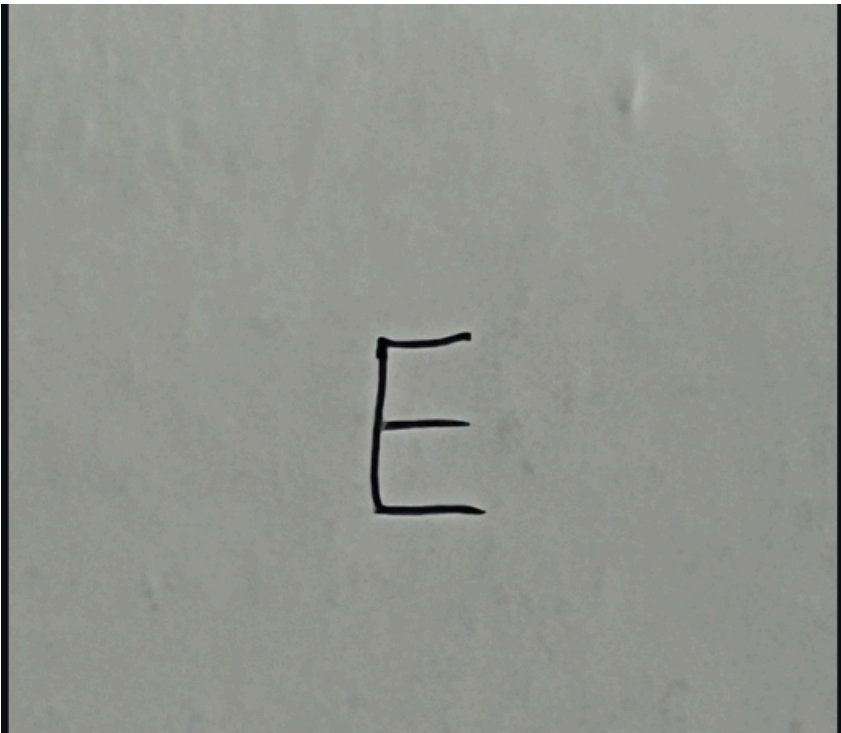
Kết quả huấn luyện

Model	Train Accuracy	Test Accuracy	Test loss	Epoch
<i>MNIST Digits</i>	99.48%	99.49%	0.0139	30
<i>Alphabet Recognition</i>	99.45%	99.47%	0.0239	15
<i>Shape Classification</i>	99.55%	99.40%	0.0341	15

- Tất cả mô hình đều đạt độ chính xác rất cao (>99%), cho thấy kiến trúc CNN và preprocessing phù hợp với từng loại dữ liệu.
- Shapes dataset giữ test accuracy ổn định, dù dữ liệu biến thiên mạnh, nhờ augmentation hiện đại giúp model generalize tốt.

Hiệu suất thực tế

Chế độ	Số ảnh test	Accuracy
Chữ số (0 - 9)	20	90%
Chữ cái (a - z)	20	88%
Shapes	20	75%



Input



Output

Tỉ lệ dự đoán đúng (accuracy) trên các bộ test ngoài dataset.

Nhận xét:

- Khả năng tổng quát hóa tốt: Model duy trì được hiệu suất ổn định trên dữ liệu thực tế, chứng minh tính ứng dụng cao trong môi trường nhiễu.
- Hiệu suất giảm do nhiễu môi trường: Sự sụt giảm Accuracy chủ yếu do sự khác biệt về nền/phông và chất lượng ảnh (bóng đổ, nhiễu sáng) so với dataset chuẩn.

Kết luận

TIỀN XỬ LÝ ẢNH (PREPROCESSING) LÀ CHÌA KHÓA

- Grayscale, Gaussian/Median filtering,otsu thresholding, đảo màu... giúp ảnh ổn định, giảm nhiễu, bảo toàn hình dạng.
- Pipeline khác nhau tùy dataset: MNIST giữ nguyên shape, Shapes xử lý mạnh để chuẩn hóa ảnh nhiễu.

XÂY DỰNG MODEL CNN HIỆU QUẢ

- Kiến trúc linh hoạt theo độ phức tạp dataset: từ 3 block (MNIST) đến 4 block + BatchNorm.
- Data augmentation mạnh cải thiện generalization, đặc biệt với Shapes và MNIST.

ỨNG DỤNG THỰC TẾ

- Tạo app nhận dạng chữ số, chữ cái và hình dạng phức tạp.
- Pipeline preprocessing + model CNN cho phép dự đoán ổn định với ảnh thực tế và nhiễu.

MỞ RỘNG

- Mở rộng thành công phạm vi: Phát triển thêm hai model mới là Chinese Handwritten và Shape Classification

Hướng phát triển

MỞ RỘNG MÔ HÌNH

- Hỗ trợ thêm chữ viết tay tiếng Việt, chữ hoa/thường, nhiều font & phong cách viết.
- Áp dụng Transfer Learning để cải thiện độ chính xác trên dữ liệu tự nhiên (ảnh chụp thật, ánh sáng kém...).

CẢI TIẾN ỨNG DỤNG

- Tích hợp OCR hoàn chỉnh: tách dòng, tách từ, nhận dạng câu chữ.
- Xây dựng phiên bản mobile (Android/iOS) tối ưu tốc độ & độ mượt.
- Cho phép người dùng tùy chỉnh mô hình, tự thêm dữ liệu và re-train nhanh.

TRIỂN KHAI THỰC TẾ

- Tích hợp vào các công cụ giáo dục: nhận dạng chữ viết tay và shapes để hỗ trợ học tập, kiểm tra, trò chơi tương tác.
- Tương tác đa nền tảng: web, mobile, tablet; cho phép nhập dữ liệu trực tiếp bằng camera hoặc bút cảm ứng.
- Hệ thống gợi ý & tự động cải thiện: mô hình học thêm từ dữ liệu người dùng, cải thiện độ chính xác theo thời gian mà không cần huấn luyện lại toàn bộ.



THANK YOU FOR WATCHING